🗺️ Flow-Wise Array Mastery Curriculum (Complete Edition)

For: Developers transitioning into DSA mastery.

Core Principle: Build intuition by increasing **Complexity of State**.

Instead of memorizing random algorithms, we progress through layers of abstraction:

1. 🟢 **Level 1: Physical Layer** (One index, basic movement)
2. 🔵 **Level 2: Logical Layer** (One index + math/wrapping)
3. 🟠 **Level 3: Multi-View Layer** (Two indices, managing relationships)
4. 🟣 **Level 4: Range Layer** (Windows, ranges, pre-computation)
5. 🔴 **Level 5: Abstract Layer** (Search spaces, partitioning)

🧭 How to use this guide

1. **Read** the Why/What/How.
2. **Trace** the "Micro-Trace" tables on paper to see the state change.
3. **Type** the C# and Python snippets from scratch (do not copy-paste).
4. **Solve** the linked problems in the Practice Set.

Rule: You "own" a topic only when you can state its **Invariant** in one sentence.

🟢 Level 1: The Physical Layer (Basic Movement)

Goal: Muscle memory for accessing memory without thinking.

1A) 🔁 Mastering the Loop

Complexity: `Time: O(N) | Space: O(1)`

1A.1 ➡️ Forward Iteration

Why: The default mode for searching, counting, and validating.

Invariant: "Indices `0` to `i-1` have been processed."

Micro-Trace: | i | Arr[i] | Action | State | | :--- | :--- | :--- | :--- | | 0 | 10 | Process | Done with 0 | | 1 | 20 | Process | Done with 0,1 |

Code:

```
# Python
for i in range(len(arr)):
    val = arr[i]
```

```
// C#
for (int i = 0; i < arr.Length; i++) {
    int val = arr[i];
}
```

1A.2 ⬅️ Backward Iteration

Why: Essential when shifting items to the right (to avoid overwriting future reads) or stack-based logic (Next Greater Element).

Invariant: "Indices `i+1` to `n-1` have been processed."

Code:

```
# Python: Start, Stop(Exclusive), Step
for i in range(len(arr) - 1, -1, -1):
    val = arr[i]
```

```
// C#
for (int i = arr.Length - 1; i >= 0; i--) {
    int val = arr[i];
}
```

1A.3 🦘 Stride Iteration (Jumping)

Why: Processing interleaved data (RGB), parity checks, or subsampling.

Code:

```
for i in range(0, len(arr), 2): # Step 2
    pass
```

1B) 📏 Boundary Control (Inclusive vs. Exclusive)

Why: Off-by-one errors are the #1 bug.

Rule: Default to **Half-Open** `[Start, End)` (Start included, End excluded).

- **Length:** `End - Start`
- **Midpoint:** `Start + (End - Start) / 2`
- **Loop:** `i < End`

Visual:

```
Indices: 0  1  2  3  4
Range [1, 4): Includes 1, 2, 3. (4 is the limit)
Length = 4 - 1 = 3.
```

1C) 🎷 Simultaneous Iteration (Lockstep)

Why: Comparing two arrays index-by-index.

How: Limit the loop to the **smaller** length of the two arrays to avoid crashes.

Code:

```
int limit = Math.Min(A.Length, B.Length);
for (int i = 0; i < limit; i++) {
    var sum = A[i] + B[i];
}
```

1D) 🧩 Edge Case Handling

Why: Crashes often happen on empty or single-element arrays. **Checklist:**

1. **Empty:** `n == 0` . (Return 0, -1, or null immediately).
2. **Single:** `n == 1` . (Logic like `arr[i] > arr[i+1]` will crash).

Code:

```
if (arr == null || arr.Length == 0) return 0;
if (arr.Length == 1) return arr[0];
```

🔵 Level 2: The Logical Layer (Index Arithmetic)

Goal: See arrays as circles and grids using math.

2A) 🔄 Cyclic / Modular Arithmetic

Complexity: `Time: O(1) per step | Space: O(1)`

Why: Circular buffers, rotating arrays, hash tables.

Invariant: "Index always stays within `[0, n-1]` ."

Formula: `index = (index + step) % length`

Visual:

```
Len = 5. Index = 4. Step = 1.
(4 + 1) % 5 = 0. (Wraps to start)
```

Code:

```
# Python (Handles negative mod correctly)
next_i = (i + 1) % n
prev_i = (i - 1 + n) % n
```

```
# C# (Need extra math for negative mod)
int nextI = (i + 1) % n;
int prevI = ((i - 1) % n + n) % n; // Essential for safety
```

2B) 🧱 2D-to-1D Mapping (Flattening)

Complexity: `Time: O(1) map | Space: O(1)`

Why: Binary Search a matrix, cache-friendly traversal.

Visual Grid:

```
Width = 4
Indices:
0  1  2  3   (Row 0)
4  5  6  7   (Row 1)
8  9  10 11  (Row 2)

Target Index: 6
Row = 6 // 4 = 1
Col = 6 % 4 = 2
Matches (Row 1, Col 2)
```

Formulas:

- `row = index / width`
- `col = index % width`
- `index = row * width + col`

2C) ➕ Offset & Relative Indexing

Why: Accessing neighbors (`i-1` , `i+1`) safely. **Rule:** Always guard the offset check. **Code:**

```
# Check Right Neighbor
if i + 1 < len(arr) and arr[i] == arr[i+1]:
    pass
```

🟠 Level 3: The Multi-View Layer (Dual Pointers)

Goal: Manage two states to optimize from $O(N^2)$ to $O(N)$.

3A) 🧹 Reader/Writer (Compaction)

Complexity: `Time: O(N) | Space: O(1)`

Why: In-place modification (filtering/removing).

Invariant: `arr[0...write-1]` contains the valid, processed result.

Micro-Trace (Move Zeros): | Step | Arr State | Read | Write | Action | | :--- | :--- | :--- | :--- | :--- | | 0 | `[0, 1, 0]` | 0 (Val 0) | 0 | Skip (Read is 0) | | 1 | `[0, 1, 0]` | 1 (Val 1) | 0 | Swap/Write. `Arr[0]=1` . Inc Write. | | 2 | `[1, 0, 0]` | 2 (Val 0) | 1 | Skip. |

Code:

```
int write = 0;
for (int read = 0; read < arr.Length; read++) {
    if (should_keep(arr[read])) {
        arr[write] = arr[read];
        write++;
    }
}
```

3B) 🦀 Converging Pointers (Pincer)

Complexity: `Time: O(N) | Space: O(1)`

Why: Finding pairs in **Sorted** arrays.

Invariant: The target pair, if it exists, is within `[Left, Right]` .

Visual:

```
Target: 10
[ 2, 4, 6, 8, 9 ]
  L           R
2 + 9 = 11 (> 10).
Sorted property implies 9 is too big for ANY number on the left.
Discard 9. Decrement R.
```

3C) 🌗 Diverging Pointers (Center Expansion)

Complexity: `Time: O(N) per center | Space: O(1)`

Why: Palindromes, "k-closest elements" (around a pivot).

Invariant: `Arr[L...R]` satisfies the symmetry condition.

Code (Palindrome):

```
def expand(l, r):
    while l >= 0 and r < len(s) and s[l] == s[r]:
        l -= 1; r += 1
    return r - l - 1 # Length
```

3D) 🐢 Fast & Slow Pointers (Tortoise & Hare)

Complexity: `Time: O(N) | Space: O(1)`

Why: Cycle detection, finding midpoints, finding duplicate numbers (when value maps to index).

Visual:

```
Fast moves 2 steps. Slow moves 1 step.
If a cycle exists, Fast will lap Slow.
If no cycle, Fast hits the end.
```

Code (Cycle Detection):

```
slow, fast = head, head
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
    if slow == fast: return True # Cycle found
return False
```

🟣 Level 4: The Range Layer (Subarrays & Windows)

Goal: Process *groups* of elements efficiently.

4A) 🐛 Fixed Sliding Window

Complexity: `Time: O(N) | Space: O(1)`

Why: Sum/Avg of every subarray of size `k`.

Invariant: `Current_Sum` represents `Arr[i-k+1...i]`.

Code:

```
curr = sum(arr[:k])
ans = curr
for i in range(k, len(arr)):
    curr += arr[i] - arr[i-k] # Add New, Remove Old
```

```
        ans = max(ans, curr)
```

4B) 🪗 Variable Sliding Window (Accordion)

Complexity: `Time: O(N) | Space: O(1)`

Why: Longest/Shortest subarray meeting criteria.

Invariant: `[Left, Right]` is the current window being evaluated.

Micro-Trace (Min Subarray Len, Sum >= 7): | Step | L, R | Window | Sum | Action | | :--- | :--- | :--- | :--- | :--- | | 1 | 0, 0 | `[2]` | 2 | Invalid (<7). Expand R. | | 2 | 0, 3 | `[2,3,1,2]` | 8 | Valid (>=7). Record Len 4. Shrink L. | | 3 | 1, 3 | `[3,1,2]` | 6 | Invalid (<7). Expand R. |

Code Template:

```
left = 0
for right in range(len(arr)):
    # 1. Expand (Ingest arr[right])
    while condition_met:
        # 2. Record Answer
        # 3. Shrink (Eject arr[left], left++)
```

4C) 🧻 Prefix Sums (1D)

Complexity: `Time: O(1) Query | Space: O(N)`

Why: Static Range Sum Queries.

Formula: `Sum(L, R) = P[R+1] - P[L]`

Visual:

```
Arr: [ 10, 20, 30 ]
Pre: [ 0, 10, 30, 60 ] (Size N+1)
Sum(1, 2) = Pre[3] - Pre[1] = 60 - 10 = 50.
```

4D) 🧱 2D Prefix Sums

Complexity: `Time: O(1) Query | Space: O(N*M)`

Why: Range Sum Queries in a Grid.

Formula:

```
Sum(r1, c1, r2, c2) = P[r2+1][c2+1] - P[r1][c2+1] - P[r2+1][c1] + P[r1][c1]
```

Code Construction:

```
P[i][j] = matrix[i-1][j-1] + P[i-1][j] + P[i][j-1] - P[i-1][j-1]
```

4E) 💉 Difference Arrays (Inverse Prefix)

Complexity: `Time: O(1) Update | Space: O(N)`

Why: Increment range `[L, R]` by `X` efficiently.

How:

1. Create `Diff` array of size `N+1`.
2. `Diff[L] += X` (Start influence).
3. `Diff[R+1] -= X` (End influence).
4. Compute Prefix Sum of `Diff` to get the final result.

4F) 🧠 Prefix Sum + HashMap

Complexity: `Time: O(N) | Space: O(N)`

Why: Find number of subarrays with `Sum == K`. (Handles negatives).

Logic: `Sum(i, j) = Pref[j] - Pref[i-1]`. We want `Pref[j] - Pref[i-1] == K`. Algebra: `Pref[i-1] == Pref[j] - K`.

Code:

```
count = 0, curr = 0
map = {0: 1} # Base case: Sum 0 exists once (empty subarray)
for x in arr:
    curr += x
    if (curr - k) in map:
        count += map[curr - k]
    map[curr] = map.get(curr, 0) + 1
```

4G) 📉 Monotonic Deque

Complexity: `Time: O(N) | Space: O(k)`

Why: Sliding Window Maximum/Minimum.

Invariant: "Deque contains indices of potential maximums. Values are sorted Descending."

Logic:

1. **Expire:** Pop front if index is out of window `(i - k)`.
2. **Kill Weak:** Pop back if `Arr[back]` is smaller than `Arr[i]` (Current is newer AND stronger).

3. **Push:** Push `i`.
4. **Answer:** `Arr[Deque.Front()]` is the max.

🔴 Level 5: The Abstract Layer (Search & Partition)

Goal: Treat array as a search space.

5A) 🧱 Partitioning (2-Way)

Complexity: `Time: O(N) | Space: O(1)`

Why: Move elements satisfying a predicate to one side.

Invariant: `[0, Boundary-1]` satisfies predicate.

5B) NL 3-Way Partitioning (Dutch National Flag)

Complexity: `Time: O(N) | Space: O(1)`

Why: Sort 3 categories (Low, Mid, High) in one pass.

Invariant:

- `0` to `Low-1` : Small (0s)
- `Low` to `Mid-1` : Medium (1s)
- `Mid` to `High` : Unknown
- `High+1` to `End` : Large (2s)

Code:

```
while mid <= high:
    if nums[mid] == 0:
        swap(low, mid); low++; mid++
    elif nums[mid] == 1:
        mid++
    else:
        swap(mid, high); high--
```

5C) 🔍 Binary Search (Classic)

Complexity: `Time: O(log N)`

Invariant: `Target` is in `[Left, Right]`.

Code:

```
while (left <= right) {
    int mid = left + (right - left) / 2;
    if (arr[mid] < target) left = mid + 1;
    else right = mid - 1;
```

```
}
```

5D) 📍 Lower/Upper Bound

Why: Find *insertion point* or *first occurrence*.

Difference: When `arr[mid] == target`, do not return.

- **Lower Bound (First):** `right = mid - 1` (Keep looking left).
- **Upper Bound (Last):** `left = mid + 1` (Keep looking right).

5E) 💉 Binary Search on Answer Space

Complexity: `Time: O(N log Range)`

Why: Minimize the maximum capacity/time.

Logic:

1. Identify Search Space (e.g., `[MaxElement, SumElements]`).
2. Write `IsFeasible(x)` function (Greedy check).
3. If `IsFeasible(mid)` is true, `Right = mid - 1` (Try harder/smaller). Else `Left = mid + 1`.

✅ Interview-Grade Debug Checklist